# Control de errores en JavaScript

Documento de apoyo · Unidad 7.2

Programar es, en buena parte, encontrar y solucionar errores. Casi todas las personas que desarrollan han pasado horas buscando un fallo. Lo peor es que a veces se detectan mucho después, porque ocurren situaciones de uso que no se habían tenido en cuenta. Este documento resume lo que vemos en clase.

## 1. Tipos de errores

| Tipo | Qué ocurre | Ejemplo |
|---|---|---|
| **Sintaxis** | Se detecta al escribir. Editores como Visual Studio Code subrayan en rojo el código erróneo. | Una llave sin cerrar o una palabra clave mal escrita |
| **Ejecución** | Solo aparece al ejecutar el código. | Llamar a una función que aún no se ha definido |
| **Lógicos** | La sintaxis es correcta, pero el programa no funciona como debería. Ninguna herramienta avisa: lo detecta el desarrollador o el usuario. | Un IVA mal calculado |
| **Del sistema** | Circunstancias fuera del control del programador. No podemos evitarlas, pero sí limitar el daño. | Fallo de red, caída de un servicio |
| **De usuario** | Acciones inesperadas del usuario. En realidad son errores lógicos por no prever esa situación. | Pedir un número y recibir un texto |

La habilidad de encontrar errores se gana con los años de experiencia, pero conviene conocer las técnicas fundamentales desde el principio.

## 2. Crear y lanzar errores propios

El intérprete de JavaScript crea errores cuando ocurren, pero también podemos crear los nuestros:

```
let miError = new Error("Se esperaba un número");
throw miError;
```

- Crear el objeto de error **no** provoca ningún error.
- Lanzar el error y provocar una **excepción** se hace con la palabra clave `throw`.

## 3. Gestionar excepciones: try…catch

```
try {
  // código que puede provocar un error
} catch (objetoError) {
  // código que se ejecuta si hay error
}
```

Si el código del `try` provoca un error, el flujo pasa al `catch` y **el resto de líneas del `try` no se ejecutan**.

### Ejemplo 1: la variable no existe

```
try {
  console.log(e);
  console.log("aquí");
} catch (error) {
  let e = 1;
  console.log(e);
}
```

La variable `e` no está declarada en el `try`, así que `console.log(e)` provoca un error. Se ejecuta el `catch` y se muestra:

```
1
```

### Ejemplo 2: la variable ya existe

```
let e = 12;
try {
  console.log(e);
  console.log("aquí");
} catch (error) {
  let e = 1;
  console.log(e);
}
```

No hay error, así que el `catch` no se ejecuta. Se muestra:

```
12
aquí
```

## 4. El bloque finally

El contenido de `finally` se ejecuta tanto si se produce la excepción como si no:

```
try {
  console.log(e);
  console.log("aquí");
} catch (error) {
  let e = 1;
  console.log(e);
} finally {
  console.log("Yo salgo siempre");
}
```

Salida:

```
1
Yo salgo siempre
```

## 5. Un ejemplo completo

```
try {
  let n = prompt("Dame un número");
  if (isNaN(n)) throw new Error("Se esperaba un número");
  console.log("Doble:", n * 2);
} catch (error) {
  console.log("Error:", error.message);
} finally {
  console.log("Fin del programa");
}
```

`error.message` contiene el texto que escribimos al crear el error.

## 6. Resumen

| Palabra | Para qué sirve |
| --- | --- |
| `try` | Contiene el código que puede fallar |

| Palabra | Para qué sirve |
| --- | --- |
| `catch` | Se ejecuta solo si hay un error |
| `finally` | Se ejecuta siempre, haya error o no |
| `throw` | Lanza un error para provocar una excepción |

## 7. Errores frecuentes al usar try…catch

- Escribir un `try` sin `catch` ni `finally`.
- Olvidar los paréntesis o las llaves del `catch`.
- Suponer que las líneas posteriores al fallo, dentro del `try`, se ejecutan.
- Declarar con `let` una variable dentro del `try` e intentar usarla fuera: solo existe dentro de su bloque.

## 8. Para repasar

1. Nombra los cinco tipos de errores y pon un ejemplo de cada uno.
2. ¿Qué diferencia hay entre crear un error y lanzarlo?
3. ¿Cuándo se ejecuta el bloque `catch`? ¿Y el bloque `finally`?